

INDIVIDUAL MODULE-WISE LAB WORKBOOK

EXPERIMENT 1

Module: Automation with Selenium

Student Name: RAHUL KUMAR SHAW

Enrollment Number: 12023002022132

Department: CST

Institution: Institute of Engineering and Managent, Newtown

Experiment 1: Automation with Selenium

Module Name and Experiment Title

Module Name: Automation with Selenium

Experiment Title: Web Application Automation and Functional Testing Using Selenium WebDriver

This experiment combines the concepts covered in the Automation with Selenium module into one integrated practical task. The experiment focuses on configuring Selenium, locating web elements, performing browser interactions, handling synchronization, validating results, and organizing automation code in a maintainable manner.

Problem Statement

Manual testing of web applications is repetitive, time-consuming, and prone to human error, especially when the same workflow must be executed across multiple test cases or browser sessions. The objective of this experiment is to automate a representative web application workflow using Selenium WebDriver. The automation should open the browser, navigate to the target website, identify web elements using reliable locators, perform user actions such as entering data and clicking buttons, validate the expected page behavior, and record the test outcome. The solution should use appropriate wait mechanisms and a structured coding approach so that the test remains stable and maintainable.

Objective

- To understand the purpose and working of Selenium WebDriver in web application automation.
- To configure the Python environment and browser driver required for Selenium execution.
- To identify and interact with web elements using suitable locators such as ID, name, CSS selector, XPath, and accessible attributes.
- To automate common browser actions such as opening a URL, entering text, clicking buttons, and navigating between pages.
- To use explicit waits and expected conditions for handling dynamic web elements.
- To validate actual results against expected results using assertions or conditional checks.
- To organize the automation workflow into reusable functions or Page Object Model components.
- To analyze the execution output and document the observations using labelled screenshots.

Tools, Software, and Concepts Used

Category	Details
Programming Language	Python
Automation Tool	Selenium WebDriver
Browser	Google Chrome or another supported browser
Development Environment	Visual Studio Code / PyCharm
Package Manager	pip
Environment	Python virtual environment (venv)
Testing Concepts	Test cases, assertions, positive testing, negative testing, and regression testing
Locator Strategies	ID, name, class name, tag name, link text, partial link text, CSS selector, and XPath

Synchronization	Implicit wait, explicit wait, WebDriverWait, and expected conditions
Design Approach	Reusable functions and Page Object Model (if implemented)
Reporting Evidence	Console output, screenshots, and execution observations

Source Code or Implementation Steps

The complete module was implemented through the following steps:

1. Install Python and create a virtual environment for the project.
2. Install Selenium using the command: `pip install selenium`.
3. Import the required Selenium modules, `WebDriver`, `By`, `WebDriverWait`, and `expected conditions`.
4. Launch the browser and configure the browser driver.
5. Open the target web application using `driver.get(URL)`.
6. Identify the required fields and controls using reliable locators.
7. Enter test data, click the required controls, and navigate through the workflow.
8. Use explicit waits to ensure that elements are visible, present, or clickable before interaction.
9. Verify the expected result using an assertion or a conditional validation.
10. Capture screenshots and close the browser using `driver.quit()`.

Representative Selenium implementation:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
wait = WebDriverWait(driver, 10)

try:
    driver.get("https://example.com")
    driver.maximize_window()

    heading = wait.until(
        EC.visibility_of_element_located((By.TAG_NAME, "h1"))
    )

    assert heading.is_displayed()
    print("Test passed: Page loaded successfully")

finally:
    driver.quit()
```

Note: Replace the sample URL, locators, and actions with the actual website and workflow used during the laboratory demonstration.

Output with Clearly Labelled Screenshots

The following screenshots should be attached as evidence of successful execution. Each screenshot must be labelled with a figure number and a short description.

Figure 1: Selenium environment and browser launch

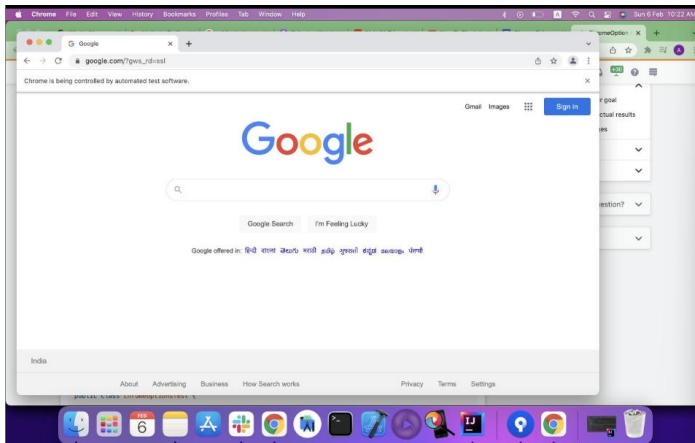


Figure 2: Target web page opened successfully

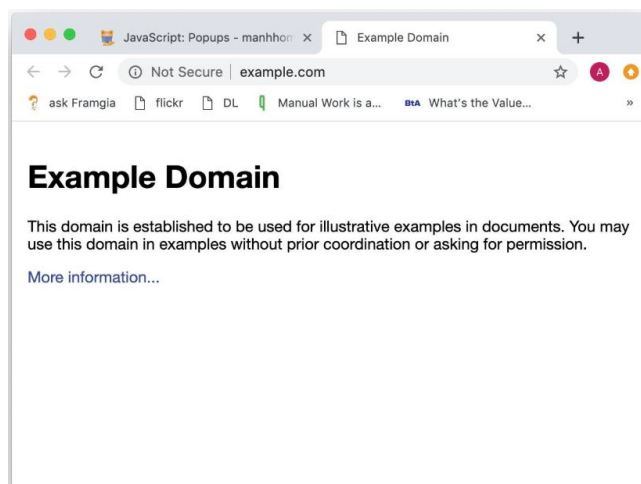


Figure 3: Web element identification and data entry

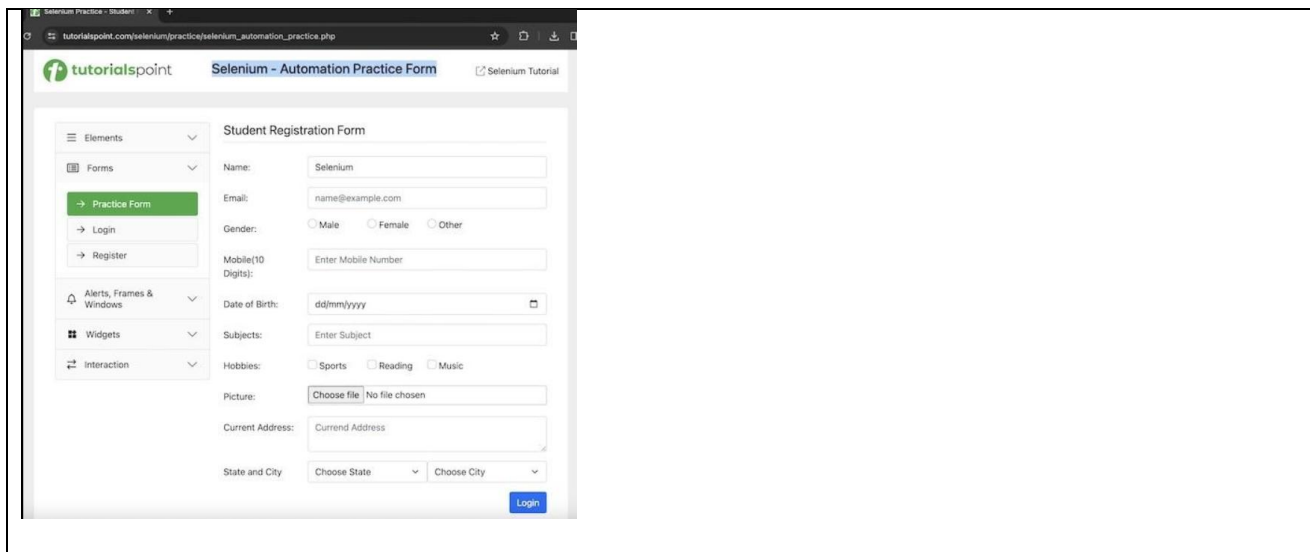


Figure 4: Button click or workflow execution

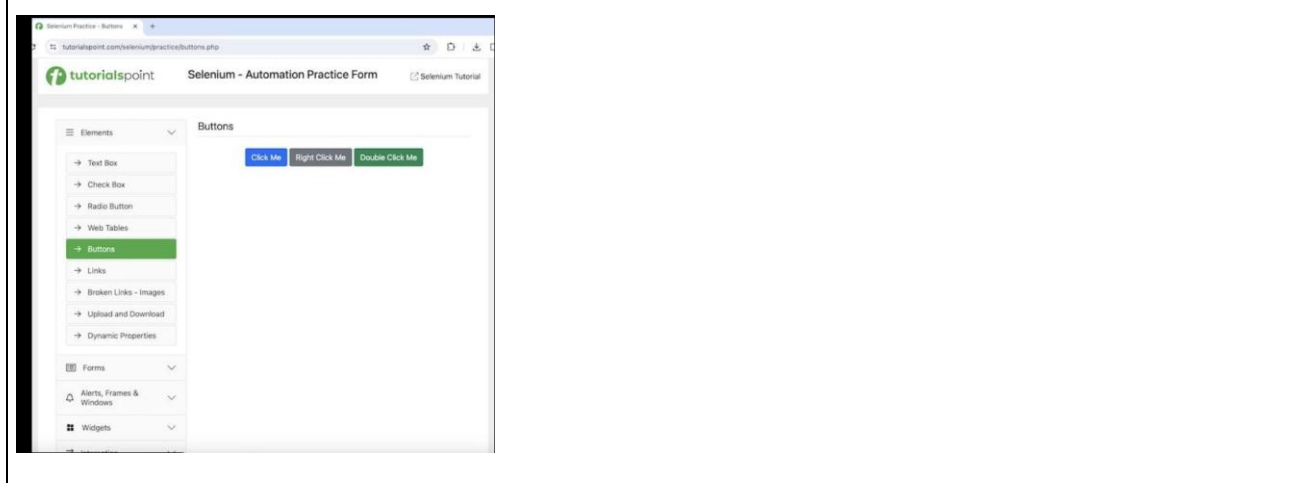
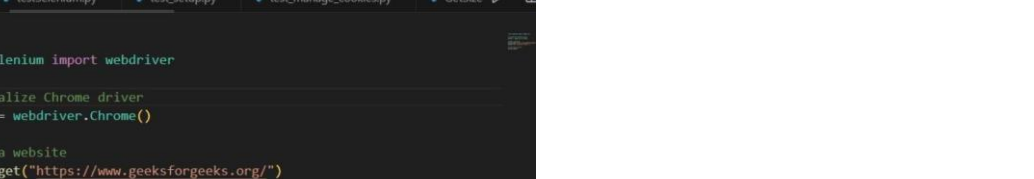


Figure 5: Validation of the expected output



Figure 6: Console output showing test execution status



```
newTest.py > ...
1
2 from selenium import webdriver
3
4 # Initialize Chrome driver
5 driver = webdriver.Chrome()
6
7 # Open a website
8 driver.get("https://www.geeksforgeeks.org/")
9 print("Test Execution Done!!")
10
11 # Close the browser
12 driver.quit()
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER
-a---- 02-06-2025 11:40 1501 test_setup.py
-a---- 04-12-2024 11:36 300 tuple.py

PS D:\gfg329\Software testing\New folder> python newTest.py

DevTools listening on ws://127.0.0.1:63550/devtools/browser/3ac74bbe-c4bf-4f80-b14a-dd7176d0c069
Test Execution Done!!
PS D:\gfg329\Software testing\New folder>
```

The Automation with Selenium module was completed as one integrated experiment. The experiment provided practical understanding of browser automation, element location, synchronization, validation, and basic test organization. Selenium WebDriver can reduce repetitive manual effort and support consistent functional testing when scripts are designed using reliable locators, suitable waits, and reusable implementation practices.